

新程序相对于 795 程序的优势与结构分析

详细汇报版：原始逻辑、模块来源、运行方式与优点

2026-04-17

汇报结构

- 1 原始 795 程序
- 2 Bowman 与 Lin 思路
- 3 新程序整体流程
- 4 AutoDL notebook 演进
- 5 新程序详细模块分析
- 6 结果与优势总结

基本定位

`references/ftl_gen/795.py` 是围绕真实实验系统构建的工程优化脚本，其目标是在既定 SLM、既定目标和既定实验链路下，稳定地产生可直接用于实验的平顶光。

- SLM 面尺寸： $N=[1024, 1272]$
- 输出面尺寸： $NT=[6876, 6876]$
- 主要依赖：`SLM_1X.py`、`CG_2.py`、`CG_new.py`
- 目标特点：以线形平顶光为主，而不是默认圆形平顶
- 运行环境：直接面向实验装置，可接相机和 SLM 回写

795 的整体运行顺序

- ① 读取实验分辨率、光学参数与目标偏移
- ② 生成入射激光包络
- ③ 在输出面构造目标振幅和目标相位
- ④ 构造加权掩模 W_{cg} / W_{99}
- ⑤ 生成初始猜测相位 `phase_guess`
- ⑥ 用 CG 优化相位
- ⑦ 保存阶段性相位和强度结果
- ⑧ 若接入实验，则通过相机测量修正目标并重新优化

核心特征

原始 795 的重点不在于形成统一的软件框架，而在于针对当前实验系统把整条链路调通。

795 的目标构造

- 原始脚本中目标振幅不是简单圆形平顶，而是通过 `slm.gaussian_line2(...)` 构造
- 其物理含义是：
 - 一个方向存在有限平顶宽度
 - 另一个方向保持高斯衰减
- 目标相位则通常使用 `slm.phase_flat(...)`，即输出区相位尽量保持平坦
- 这一目标定义与原始实验目标强绑定，因此更偏“线形平顶实验脚本”，而不是通用全息优化器

795 的加权区域设计

- 原始脚本先通过 `slm.flat_top_round(...)` 构造一个目标区域掩模
- 再通过 `slm.weighting_value(...)` 形成 W_{cg}
- 同时根据目标强度的前 99% 区域构造 W_{99}
- 最后令 $W_{cg}=W_{99}$ ，把优化重点集中在真正关心的区域

意义

这一步反映出 795 的工程思想：不追求输出平面的全局一致，而是优先保证实验中真正使用的有效区域。

795 的初始相位与传播模型

- 初始相位由 `phase_guess(...)` 生成
- 其中显式包含：
 - 线性倾斜项：控制傅里叶面偏移
 - 二次曲率项：控制包络尺度
 - 环形项：辅助控制整体分布
- 传播模型由 `SLM_1X` 和脚本中的 Fourier operator 共同完成
- 在每次传播后，脚本会显式计算输出振幅、相位、overlap 和效率

795 的优化方式

- 并不是只有一个 cost function
- 原始脚本里至少包含三类思路：
 - ① 加权区域内的振幅平方误差
 - ② 基于 overlap 的保真度误差
 - ③ overlap 与效率惩罚组合的目标函数
- 优化器主要使用：
 - CG_2.CG
 - CG_new.CG
- 运行方式通常是短步数预优化 + 较长步数精修

795 的实验闭环

- 原始脚本可以直接调用 `IDS_Camera.GetImage()`
- 对采集图像进行裁剪、求质心、修正目标
- 再重新执行优化
- 最终通过 `slm_play.update(...)` 把新相位写回 SLM

结论

原始 795 是“目标构造 + 加权设计 + 共轭梯度 + 实验反馈”一体化的实验工程方法。

795 的优点与局限

优点

- 最贴近真实实验
- 目标与权重设计经验充分
- 参考相位具备直接实验价值
- 可以接入相机闭环

局限

- 对具体实验条件绑定很强
- 泛化能力不足
- 难以统一比较不同路线
- 难以直接接入现代训练流程

Bowman: 参考路线的定位

我对 Bowman 的基本理解

Bowman 路线的核心，不是“某个固定 CG 代码”，而是把全息问题视作一个**显式传播模型下的直接物理优化问题**：

- 目标场是明确给定的
 - 傅里叶传播模型是明确给定的
 - 代价函数直接定义在输出场 fidelity、phase 和 efficiency 上
 - 最终通过数值优化直接更新 SLM phase
-
- 这条路线的强项是：物理意义清晰、结果可解释、最终收尾稳定
 - 它的短板是：对初始化比较敏感，直接从较差初值出发时成本较高

Bowman: 源码中的落点与当前吸收方式

可对应的源码/材料

- references/Bowman/*.pdf: 思想来源
- references/ftl_gen/795.py: 原始实验脚本里的目标/传播/优化工程经验
- ai_holography/hybrid.py: 当前系统中的 bowman_cg_refine(...)
- ai_holography/scripts/compare_bowman_ai.py: Bowman / AI / hybrid 对比入口

当前系统如何吸收 Bowman

- 保留“显式傅里叶传播 + 直接物理目标函数优化”的主线
- 在 hybrid.py 中把收尾拆成:
 - efficiency_overlap
 - composite
 - phase_priority
- 说明 Bowman 在当前系统里承担的是: **最终物理收尾机制**

Lin: 参考路线的定位

我对 Lin 的基本理解

Lin 路线的核心，不是“让 CNN 直接输出最终 hologram”，而是先把问题改写到**更适合学习的表示域**：

- 用位置域 amplitude / phase 作为监督对象
 - 输入目标振幅、目标相位和任务相关 mask
 - 再通过 FFT / 逆变换恢复 hologram 初值
-
- 这条路线的强项是：能学习“什么样的初始化更容易成功”
 - 它的短板是：单独作为最终求解器时，物理一致性和稳定性通常不如后端物理优化

Lin: 源码中的落点与当前吸收方式

可对应的源码/材料

- `references/lin/*.pdf`: Lin 论文来源
- `lin2025_holography/README.md`: 位置域预测主线说明
- `lin2025_holography/model.py`: Lin2025HologramNet
- `lin2025_holography/training.py`: teacher distillation + flat-top metric training

当前系统如何吸收 Lin

- 保留“位置域预测，再转 hologram”的核心思想
- 训练时不只做 supervised loss, 还叠加:
 - teacher phase distillation
 - flat-top metric loss
- 说明 Lin 在当前系统里承担的是: **学习型初始化器**

为什么当前系统最终采用 Bowman + Lin 的混合路线

如果只有 Bowman

- 物理一致性强
- 最终结果稳
- 但初值敏感，搜索成本高

如果只有 Lin

- 初始化快
- 可以从数据中学到统计规律
- 但单独作为最终解时不够稳

当前混合路线

- 先用 Lin 学到更优初值
- 再用 Bowman 风格物理优化做最终收尾
- 中间由 runner 负责 route 选优
- 最后由 benchmark 负责统一比较

一句话

Lin 解决“从哪里开始更好”，Bowman 解决“最后怎样收得更稳、更符合物理约束”。

四条路线的角色对照：795 / Bowman / Lin / 当前混合系统

项目	795	Bowman	Lin	当前混合系统
核心定位	实验导向脚本与参考相位来源	直接物理目标函数优化	学习型初始化器	统一 route + init + hybrid 的完整系统
主要强项	最贴近真实实验，经验充分	物理一致性强，结果可解释	能学到更优初始化规律	同时吸收实验先验、学习先验和物理收尾
主要短板	条件绑定强，难统一比较	对初值敏感，成本较高	单独作为最终解时不够稳	结构更复杂，对配置一致性要求更高
解决的问题	给出已验证的目标/相位工程经验	解决“最后怎么收得稳”	解决“从哪里开始更好”	把前面三者接成可复现主线
在当前代码中的对应	references/ftl_gen/795.py	ai_holography/hybrid.py	lin2025_holography/model_runner.py + pipeline.py + benchmark	
当前答辩里的推荐表述	历史实验先验与参考基线	最终物理收尾机制	学习型初始化器	当前真正要交付和解释的主线

新程序的总体设计目标

- 保留 795 的实验先验
- 保留 Bowman/hybrid 的物理一致性
- 引入 Lin2025 风格的学习型初始化
- 建立统一 benchmark 和分析框架
- 建立远程训练、本地分析的可迭代工作流

总体思路

新程序不是简单替换原始方法，而是把 795、Bowman 和 Lin 的优势整合为一条可持续迭代的混合路线。

新程序整体运行流程

目标场构造 → 参考相位与 teacher 管理 → 初始化候选生成 → 候选选优

→ phase-only correction → 多分辨率 refine → Bowman-style hybrid polish → 统一 benchmark/分析

- 旧程序更像单链路工程脚本
- 新程序更像模块化系统

新程序的目录结构

references/ftl_gen

- 原始 795 脚本
- 参考相位
- 原始 CG 模块
- colab_local_split: 远程训练、本地分析入口
- report_package_20260331: 汇报材料与结果整理

ai_holography

- 目标构造
- 传播与损失
- runner 与 hybrid polish

lin2025_holography

- 数据集
- 模型
- 训练与推理

版本演进总览

阶段	含义
v2-v10	最早的 AutoDL 自包含化搭建期
v11-v18	legacy task / flat-top 调参与训练稳定期
v19-v30	line-flat 主线形成期
v31-v39	主线稳定前的过渡扫参与整合期
v40-v50	集成 notebook + 历史表出现期
v51-v58	高分辨率 / line-flat / 结构一致性修复期

关键拐点

- v30: 第一代强主线骨架
- v42a: reference-floor 单变量修复
- v43: checkpoint 路径与权重隔离
- v53: 高分辨率 line-flat 的效率修复
- v56: 结构一致性修复

已知元数据漂移

- v20 标题误写成 v21
- v50-v52 仍沿用 v44 标题/说明
- v54-v55 仍沿用 v53
- v57 仍沿用 v56

v2-v18 / v19-v39：从可运行到主线成形

早期搭建与训练稳定化

版本	核心变化
v2-v8	自包含 notebook 骨架逐步成形，写源码/训练/导出/benchmark 被放进同一 workflow
v9	795 参考相位生成改走 CPU，避开 AutoDL 上 FFT / cuFFT 报错
v11	OOM 回退策略改成先降 batch，不先降模型容量
v14-v15	参考目标改成带 sx/sy 的 line-flat，teacher 池也收紧到 line-flat 任务
v16-v18	继续强化 hybrid teacher 与 phase imitation，为后续主线做准备

line-flat 主线形成

版本	核心变化
v19	第一次出现较像样的 line-flat 指标，证明 learned init + hybrid 路线可行
v23	首次把 795 / hybrid / balanced / quality / feedback_proxy 放进同一 benchmark
v25	结果查看不再截断长 JSON，高分辨率 1024x1272 导出链成熟
v28-v29	主排序切到 core-phase-first，feedback_proxy 明确标成 exploratory
v30	形成最强可复用的 quality family，是后续所有集成版的算法主锚点
v33-v39	围绕 v30 做窄扫参、低效率 basin 诊断、历史表整理，最终确认应回到 v30 主线

v40-v50: 集成 notebook 与历史表阶段

版本	主变化	学到的结论	对当前主线的意义
v40	把 quality sweep 收回到真正赢出的 baseline + best_hybrid_polish 附近	集成版的重点应是收束而不是扩家族	直接引向后续干净基座
v41	先修 reference phase 与 ROI 覆盖, 再做局部 quality sweep	结构问题会直接污染路线判断	为单变量 ablation 铺路
v42a	只改 Cell5 参考相位生成的 efficiency_floor=0.05	reference-floor 修复可单独验证	关键拐点之一
v42b	只改 ROI 覆盖, 不动 floor/base cfg	ROI 扩张本身不足以成为默认主线	只保留为历史注释
v43	checkpoint 从共享 wider_net 拆到独立路径	代码版本与权重版本必须隔离	关键拐点之一
v44	集成 v30 + v42a + v43, 形成干净基座	有了可长期迭代的统一底座	集成阶段核心锚点
v45-v47	结果页、历史表、图片摆放持续清理; v46 还加入 phasor / TV / teacher annealing 等升级	文档可读性与算法试验都开始重要	v46 是重要算法分支, v45/v47 主要修展示
v48-v49	基本沿用 v47, 但 Change Log 和标题开始滞后	不能再只靠首段说明判断真实改动	版本史必须人工重建
v50	基于 v44 做 balanced 权重配置, 但元数据回退到 v44	core uniformity 变好, 效率却因旧 floor 塌到约 11%	直接引出 v51-v53 修复链

v51-v58：高分辨率、结构一致性与当前推荐版本

版本	主变化	学到的结论	当前判断
v51	修中文编码、把 route 图片移到最后一个单元；标题仍写 v44	文案修复不能解决低效率根因	过渡版
v52	SLM 提到 1024x1024，加入 line_flat_top_target；标题仍写 v44	高分辨率和 line-flat 能跑，但效率仍被旧 floor 锁死	为 v53 提供直接问题定义
v53	把 efficiency_floor 从 0.05 提到 0.80，并提高 efficiency 权重	高分辨率 + line-flat 重新回到可用区间	关键拐点之一
v54-v55	延续 v53 分支复跑，但标题/说明仍沿用 v53	元数据漂移继续累积，结构问题比算法问题更显眼	作为 v56 的前奏
v56	去重复写文件循环、统一 floor 与 route score、训练 floor 提到 0.30、output_size 提到 4096	当前主要风险从算法转到结构与一致性	关键拐点之一
v57	基于 v56 继续整理，但标题仍写 v56，导出和 summary 路径仍未完全对齐	生成器与成品 notebook 已经不同步	不是稳定交付版
v58	保留 v56 主线，补回真实导出单元、统一 summary.json 路径、补 best_route_reason、修运行目录	当前更像结构/一致性稳定版，而不是新的算法代际	当前推荐同步进汇报包的附件

明确结论

当前推荐交付 notebook = v58；当前算法关键拐点 = v30 / v42a / v43 / v53 / v56。

模块 1: 目标与参考相位模块

对应文件

- `ai_holography/targets.py`
- `ai_holography/references.py`

参考来源

- 目标构造思路参考 `795.py` 中的目标振幅、目标相位和加权区域定义
- 参考相位读取直接继承自 `795_*.npy` 这一工程资产

如何运行

- 由 `pipeline.build_problem()` 自动调用
- 也可由 `benchmark` 脚本通过 `apply_795_reference_metadata(...)` 自动适配

优点

- 统一了目标定义
- 把参考相位变成可管理、可比较的先验来源

模块 2: 传播与损失模块

对应文件

- `ai_holography/propagation.py`
- `ai_holography/losses.py`

参考来源

- 传播建模和直接优化思路主要参考 Bowman
- 多目标损失中的 `overlap`、`uniformity`、`phase` 和 `efficiency` 则综合了 Bowman 与当前实验需求

如何运行

- 在训练阶段被 `PhaseInitNet` 和 `Lin2025` 训练调用
- 在推理阶段被 `refine` 和 `hybrid polish` 反复调用

优点

- 物理意义清晰
- 可以统一衡量核心区均匀性、核心区等相位和效率

模块 3: AIHolographyPipeline

对应文件

- `ai_holography/pipeline.py`

参考来源

- 保留 Bowman 的物理传播与优化主线
- 引入 AI-first 重构思想，把目标构造、初始相位预测和多分辨率 refine 统一进同一 pipeline

如何运行

- 由 `HybridHolographyRunner` 调用
- 核心步骤包括：
 - `build_problem()`
 - `predict_initial_phase()`
 - `refine()`

优点

- 把原始分散逻辑重组为统一流程
- 易于插入新初始化器、校正项和不同 profile

模块 4: HybridHolographyRunner

对应文件

- ai_holography/runner.py

参考来源

- 借鉴 795 的参考相位和 warm start 思想
- 借鉴 Lin 的学习型初始化思路
- 保留 Bowman 的最终物理收尾

如何运行

- 主入口类: HybridHolographyRunner
- 主要步骤:
 - ① 生成 neural/reference/lin2025/warm_start 候选
 - ② 估计 overlap 和 init score
 - ③ 选出更优初始化
 - ④ 执行 phase-only correction
 - ⑤ 调用 refine 和 hybrid polish

优点

- 解决了 Bowman 对初值敏感的问题
- 把不同来源的初始化统一到一套选优机制中

模块 5: Bowman-style hybrid polish

对应文件

- ai_holography/hybrid.py

参考来源

- 直接继承 Bowman 的共轭梯度物理优化思想

如何运行

- 主函数: `bowman_cg_refine(...)`
- 内部包含三种阶段:
 - ① `efficiency_overlap` 阶段
 - ② `composite` 阶段
 - ③ `phase_priority` 阶段

优点

- 物理一致性最强
- 是当前系统最终质量的核心保障

模块 6: Lin2025 数据集与 teacher 策略

对应文件

- `lin2025_holography/dataset.py`

参考来源

- Lin 原文中的“位置域输入、位置域监督”思想
- 795 参考相位
- 当前系统自己跑出的 hybrid 高质量结果

如何运行

- 训练时通过 `create_dataset(...)` 创建数据集
- 当前主要使用 `FlatTopReferenceDataset`
- teacher 来源可在 `flat_top` / `round_top`、795 / hybrid 之间切换

优点

- 把经验参考与物理最优结果同时纳入 teacher
- 允许持续通过 benchmark 结果反哺训练

模块 7: Lin2025 网络结构

对应文件

- `lin2025_holography/model.py`

参考来源

- Lin 原文中“先在位置域预测，再经 FFT 得到 hologram”的核心思想

如何运行

- 核心类: `Lin2025HologramNet`
- 输入:
 - 目标振幅
 - 目标相位
 - core mask
 - ROI mask
- 输出:
 - 位置域振幅
 - 位置域相位
- 最后通过 `position_to_hologram(...)` 变成全息初值

优点

- 不直接硬学频域相位，训练更稳定
- 更适合和物理后端配合

模块 8: Lin2025 训练循环

对应文件

- `lin2025_holography/training.py`

参考来源

- Lin 的 supervised 框架
- 当前平顶光任务特有的核心区质量指标

如何运行

- 主入口: `train_lin2025_model(cfg)`
- 训练损失由五部分组成: 振幅监督、相位监督、hologram 场误差、teacher phase distillation、flat-top 质量损失
- 输出三个主要 checkpoint: `best_supervised`、`best_quality`、`best_hybrid_polish`

优点

- 不再只凭经验选相位
- 允许通过训练不断提升初始化质量

模块 9：统一 benchmark

对应文件

- `ai_holography/scripts/benchmark_lin2025_hybrid.py`

参考来源

- 参考 795 和 Bowman 的历史基线
- 用统一指标比较多条路线

如何运行

- 直接运行：
 - `python ai_holography/scripts/benchmark_lin2025_hybrid.py`
- 输出路线：
 - `795_only`
 - `hybrid_only`
 - `lin2025_only`
 - `lin2025_plus_hybrid_balanced`
 - `lin2025_plus_hybrid_quality`

优点

- 可以清楚回答“相对 795 到底提升了多少”
- 可以比较不同路线的 trade-off

模块 10：远程训练与本地分析

对应文件

- colab_local_split/train_lin2025_autodl_lineflat_highres_export_v58.ipynb
- colab_local_split/analyze_lin2025_checkpoint_local.py

参考来源

- 工程上为了解决 GPU 训练与本地统一分析的分工问题

如何运行

- ① 在远程平台训练并导出 checkpoint bundle
- ② 下载回本地放入新的 checkpoint 目录
- ③ 本地运行分析脚本，自动比较 official best 并给出调参建议

优点

- 训练和分析解耦
- 便于持续筛选更优 checkpoint

结果对比

路线	核心区均匀性损失	核心区相位平坦度	效率
795_only	0.02897	0.02716	0.25359
hybrid_only	0.03093	0.00302	0.85184
lin2025_plus_hybrid_balanced	0.03003	0.00192	0.82033
lin2025_plus_hybrid_quality	0.02693	0.00408	0.79291
AutoDL best supervised quality	0.02640	0.00134	0.78949

相对于 795 的优势

- 不再依赖单一经验参考相位
- 可以统一比较多条路线
- 可以通过训练持续提升初始化质量
- 在核心区等相位和效率上明显优于 795_only
- 已形成标准化 benchmark 和 checkpoint 选优流程

相对于 Bowman/hybrid 的价值

- 不替代 Bowman，而是补强其最大短板：初始化
- 让后端物理优化更容易进入高质量区域
- 在若干结果中，已经找到优于纯 Bowman 基线的 Pareto 点
- 保留物理一致性，避免神经网络单独输出不稳定的问题

当前不足

- 默认目标仍偏圆形平顶，与原始线形平顶不完全一致
- 默认主流程仍以仿真闭环为主，尚未完全恢复真实相机闭环
- 分辨率尚未完全对齐实验 SLM
- checkpoint 仍存在波动
- 质量与效率之间仍有 trade-off

核心结论

新程序相对于 795 的最大改进，不是单一算法更换，而是把 795 的实验先验、Bowman 的物理优化主线和 Lin 的学习型初始化思想整合成了一套完整系统。

- 795 负责提供经验参考与实验先验
- Lin2025 负责提供更优初始化
- Bowman/hybrid 负责最终物理收尾
- benchmark 与本地分析负责统一评估与持续迭代
- 当前推荐交付 notebook = v58; 真正决定当前算法方向的关键版本 = v30 / v42a / v43 / v53 / v56